

Python Programming

Advanced Self-Test — Hard Level

ETCS220 · B.Tech CSE · SY Sem IV · DES Pune University · Pattern 2024R1

Name:		Roll No.:	
Date:		Total / 284:	

Unit	Topic	Sections	Max Marks	Your Score
1	Introduction to Python	A-F	71	
2	Loops and Data Structures	A-F	71	
3	Functions and Strings	A-F	71	
4	File Handling and OOP	A-F	71	
	TOTAL		284	

Difficulty	This test is harder than exam level. Every question tests deeper understanding.
MCQ	8 questions × 2 marks. Multiple concepts tested per question. No negative marking.
Output Predict	Write the exact output. If an exception is raised, write ERROR and name it.
Code Trace	Fill in variable values at each step. Show intermediate states, not just final values.
Fill Blanks	One word/expression per blank. Spell correctly — method names must be exact.
Error Fix	State the bug clearly, then write the full corrected code (not just the fixed line).
Write Programs	All 3 questions are compulsory. Comment your key logic steps.
Answer Key	At the back of this booklet. Complete each unit fully before checking.

UNIT 1 — Introduction to Python

Advanced difficulty · 8 MCQs · 6 Output Predictions · 2 Code Traces · 6 Fill-in-Blanks · 3 Error Fixes · 3 Write Programs

Section	Marks	Your Score	Remarks
A — MCQ	16		
B — Output Predict	12		
C — Code Trace	10		
D — Fill Blanks	6		
E — Error Fix	12		
F — Write Programs	15		
TOTAL	71		

Part A — Multiple Choice [8 × 2 = 16 marks]

Circle the correct answer. No partial credit.

Q1. What is the output of <code>bool("")</code> or <code>bool(0)</code> or <code>bool([])</code>? (A) True (B) False (C) None (D) Error Circle your answer: (A) (B) (C) (D)	Q2. Which expression evaluates to True? (A) <code>not not 0</code> (B) <code>not not ""</code> (C) <code>not not []</code> (D) <code>not not 1</code> Circle your answer: (A) (B) (C) (D)
Q3. What does <code>print(0.1 + 0.2 == 0.3)</code> print? (A) True (B) False (C) 0.3 (D) Error Circle your answer: (A) (B) (C) (D)	Q4. What is the value of x after: <code>x = 5; x /= 2; x **= 2</code>? (A) 6 (B) 4 (C) 2 (D) 9 Circle your answer: (A) (B) (C) (D)
Q5. Which is the correct way to swap two variables without a temp variable? (A) <code>a,b = b,a</code> (B) <code>a = b; b = a</code> (C) <code>swap(a,b)</code> (D) <code>a ^= b ^= a</code> Circle your answer: (A) (B) (C) (D)	Q6. What is the output of <code>print(type(1/1))</code>? (A) <code><class 'int'></code> (B) <code><class 'float'></code> (C) <code><class 'double'></code> (D) Error Circle your answer: (A) (B) (C) (D)
Q7. What is the output of <code>x=10; print(x>5>3)</code>? (A) True (B) False (C) Error (D) TrueTrue Circle your answer: (A) (B) (C) (D)	Q8. Which statement about <code>pass</code> is correct? (A) Exits the loop (B) Skips one iteration (C) Does nothing — placeholder (D) Returns None Circle your answer: (A) (B) (C) (D)

Part B — Predict the Output [6 × 2 = 12 marks]

Write exactly what each snippet prints. Write ERROR if it raises an exception.

Code Snippet	Your Output	Expected
<pre>x = 15 print(x > 10 > 5)</pre>		
<pre>a = b = c = 5 print(a, b, c)</pre>		
<pre>x = 10 x, y = x + 1, x - 1 print(x, y)</pre>		
<pre>print(int(True) + int(False) + 1)</pre>		
<pre>x = '5' y = int(x) ** 2 print(type(y), y)</pre>		
<pre>x = 5 if x % 2: print("odd") else: print("even")</pre>		

Part C — Code Execution Trace [2 × 5 = 10 marks]

Fill in the variable table step by step and write the final output.

Q1. Trace the execution — fill in the variable table and final output.

<pre>x = 1 for i in range(4): x = x * (i + 1) print(x)</pre>			
Step	i	x	Output

Q2. Trace the execution — fill in the variable table and final output.

```
a, b = 6, 4
while b != 0:
    a, b = b, a % b
print(a)
```

Step	a	b	Output

Part D — Fill in the Blanks [6 × 1 = 6 marks]

Complete each statement with the correct term, symbol, or value.

- 1. The operator _____ gives the remainder of division.
- 2. In Python, _____ is the result of 10 / 4 (not 10 // 4).
- 3. Python uses _____ instead of curly braces to define code blocks.
- 4. The _____ function converts a string to an integer.
- 5. A chained comparison like 1 < x < 10 is equivalent to _____.
- 6. The built-in _____ function returns the type of a variable.

Part E — Find & Fix the Error [3 × 4 = 12 marks]

Identify the bug(s) and write the corrected version of the code.

1. This program should compute x squared but crashes. Find and fix all bugs.

```
x = input("Enter number: ")
result = x ^ 2
print("Square is: " result)
```

Bug identified:

Corrected code:

2. The if-elif chain should print grade but always prints "Fail". Fix it.

```
marks = int(input("Marks: "))
if marks > 90:
    print("A")
if marks > 75:
    print("B")
if marks > 60:
    print("C")
else:
    print("Fail")
```

Bug identified:

Corrected code:

3. Fix the infinite loop.

```
n = 10  
while n > 0:  
    print(n)
```

Bug identified:

Corrected code:

Part F — Write Programs [3 × 5 = 15 marks]

Attempt ALL three questions. Write clean, commented code.

Q1. Write a program that takes a positive integer n and prints the multiplication table of n from 1 to 10, formatted as " $n \times i = \text{result}$ ".

Hint: Use a for loop with `range(1, 11)`. f-strings or `format()` make the output clean.

Q2. Write a program to check if a number is a **perfect number**. A perfect number equals the sum of its proper divisors (e.g. $6 = 1+2+3$).

Hint: Proper divisors are all divisors excluding the number itself. Use % to find divisors.

Q3. Write a program that prints all prime numbers between 1 and 100 using nested loops. Count them too.

Hint: A prime has no divisors other than 1 and itself. Check divisibility from 2 to $\sqrt{n}$.

UNIT 2 — Loops and Data Structures

Advanced difficulty · 8 MCQs · 6 Output Predictions · 2 Code Traces · 6 Fill-in-Blanks · 3 Error Fixes · 3 Write Programs

Section	Marks	Your Score	Remarks
A — MCQ	16		
B — Output Predict	12		
C — Code Trace	10		
D — Fill Blanks	6		
E — Error Fix	12		
F — Write Programs	15		
TOTAL	71		

Part A — Multiple Choice [8 × 2 = 16 marks]

Circle the correct answer. No partial credit.

Q1. What is the output of `L = [1, 2, 3]; L[1:2] = [10, 20]; print(L)`?

(A) [1,10,20,3] (B) [1,[10,20],3] (C) [10,20,3] (D) Error

Circle your answer: (A) (B) (C) (D)

Q2. What does `list(zip([1, 2], [3, 4], [5, 6]))` produce?

(A) [[1,3,5],[2,4,6]] (B) [(1,3,5),(2,4,6)] (C)

[(1,2),(3,4),(5,6)] (D) Error

Circle your answer: (A) (B) (C) (D)

Q3. Which operation on a frozenset raises an `AttributeError`?

(A) `len(fs)` (B) `fs | {1}` (C) `fs.add(4)` (D) `4 in fs`

Circle your answer: (A) (B) (C) (D)

Q4. What is the time complexity of checking membership in a set (`x in s`)?

(A) $O(n)$ (B) $O(\log n)$ (C) $O(1)$ average (D) $O(n^2)$

Circle your answer: (A) (B) (C) (D)

Q5. What does `{k:v for k,v in d.items() if v > 5}` do?

(A) Filters keys (B) Filters by value > 5 (C) Inverts the dict (D) Sorts the dict

Circle your answer: (A) (B) (C) (D)

Q6. What is the output of `d = {}; d[1] = d[1] + 1 if 1 in d else 1; print(d)`?

(A) Error (B) `{1: 0}` (C) `{1: 1}` (D) `{1: 2}`

Circle your answer: (A) (B) (C) (D)

Q7. Which list method has $O(1)$ complexity?

(A) `list.insert(0, x)` (B) `list.pop()` (C) `list.remove(x)` (D) `list.index(x)`

Circle your answer: (A) (B) (C) (D)

Q8. What does `sorted({3, 1, 4, 1, 5, 9})` return?

(A) Error — sets are unordered (B) `{1,3,4,5,9}` (C) `[1,3,4,5,9]` (D) `(1,3,4,5,9)`

Circle your answer: (A) (B) (C) (D)

Part B — Predict the Output [6 × 2 = 12 marks]

Write exactly what each snippet prints. Write ERROR if it raises an exception.

Code Snippet	Your Output	Expected
<pre>L = [1,2,3,4,5] print(L[::-2])</pre>		
<pre>a = [1,2] b = a b.append(3) print(a)</pre>		
<pre>d = {"a":1,"b":2} d.update({"b":10,"c":3}) print(sum(d.values()))</pre>		
<pre>L = [3,1,2] print(sorted(L), L)</pre>		
<pre>A={1,2,3};B={2,3,4} print(len(A^B))</pre>		
<pre>L = [[1,2],[3,4]] print(L[1][0])</pre>		

Part C — Code Execution Trace [2 × 5 = 10 marks]

Fill in the variable table step by step and write the final output.

Q1. Trace the execution — fill in the variable table and final output.

<pre>nums = [5, 3, 8, 1, 9, 2] maxi = nums[0] for n in nums: if n > maxi: maxi = n print(maxi)</pre>			
Step	n	maxi	Output

Part D — Fill in the Blanks [6 × 1 = 6 marks]

Complete each statement with the correct term, symbol, or value.

1. To copy a list without aliasing, use `b = a. _____` or `b = a[ : ]`.
2. `dict.items()` returns a view of _____ pairs.
3. The _____ method removes and returns the last element of a list.
4. A set is _____, meaning you cannot index or slice it.
5. `[x for x in range(10) if x%3==0]` produces _____.
6. To merge two dicts in Python 3.9+, use the _____ operator.

Part E — Find & Fix the Error [3 × 4 = 12 marks]

Identify the bug(s) and write the corrected version of the code.

1. The code should print a copy of the list doubled but has an aliasing bug.

```
original = [1, 2, 3]
copy = original
copy.append(4)
print("original:", original)
print("copy:", copy)
```

Bug identified:

Corrected code:

2. This dictionary word-frequency counter crashes. Fix it.

```
words = ["cat", "dog", "cat", "bird", "dog", "cat"]  
freq = {}  
for w in words:  
    freq[w] += 1  
print(freq)
```

Bug identified:

Corrected code:

3. Fix: the program should find common elements between two lists efficiently.

```
A = [1,2,3,4,5]  
B = [3,4,5,6,7]  
common = []  
for x in A:  
    for y in B:  
        if x == y:  
            common.append(x)  
print(common)
```

Bug identified:

Corrected code:

Part F — Write Programs [3 × 5 = 15 marks]

Attempt ALL three questions. Write clean, commented code.

Q1. Write a program that reads a sentence from the user and prints a dictionary showing the frequency of each word (case-insensitive, ignore punctuation).

Hint: Use `.lower()` and `.replace()` to normalize. Use `dict.get(word, 0) + 1` pattern.

Q2. Write a program that uses list comprehension to generate a 5x5 multiplication table stored as a 2D list, then prints it formatted.

Hint: Nested list comprehension: `[[i*j for j in range(1,6)] for i in range(1,6)]`

Q3. Write a program that takes a list of integers and partitions them into three lists: negatives, zeros, and positives — using a single pass through the list.

Hint: Use three separate lists and a single for loop. Do NOT use filter() for this question.

UNIT 3 — Functions and Strings

Advanced difficulty · 8 MCQs · 6 Output Predictions · 2 Code Traces · 6 Fill-in-Blanks · 3 Error Fixes · 3 Write Programs

Section	Marks	Your Score	Remarks
A — MCQ	16		
B — Output Predict	12		
C — Code Trace	10		
D — Fill Blanks	6		
E — Error Fix	12		
F — Write Programs	15		
TOTAL	71		

Part A — Multiple Choice [8 × 2 = 16 marks]

Circle the correct answer. No partial credit.

Q1. What is the output of <code>f = lambda *a: sum(a); print(f(1,2,3,4))</code>? (A) Error (B) 10 (C) [1,2,3,4] (D) None Circle your answer: (A) (B) (C) (D)	Q2. What does <code>list(map(str, [1,2,3]))</code> return? (A) ['1','2','3'] (B) [1,2,3] (C) ('1','2','3') (D) Error Circle your answer: (A) (B) (C) (D)
Q3. How many times does <code>fib(5)</code> call itself (including the initial call) using naive recursion? (A) 5 (B) 9 (C) 15 (D) 25 Circle your answer: (A) (B) (C) (D)	Q4. What is the output of <code>"python"[1::2]</code>? (A) yh (B) pto (C) yto (D) phn Circle your answer: (A) (B) (C) (D)
Q5. Which regex pattern matches a valid Indian mobile number (starts with 6-9, 10 digits)? (A) <code>r"\d{10}"</code> (B) <code>r"[6-9]\d{9}"</code> (C) <code>r"[0-9]{10}"</code> (D) <code>r"[6789]\d{10}"</code> Circle your answer: (A) (B) (C) (D)	Q6. What is a closure in Python? (A) A function inside a class (B) An inner function that captures variables from its enclosing scope (C) A lambda with no arguments (D) A function that returns None Circle your answer: (A) (B) (C) (D)
Q7. What does <code>re.sub(r"\s+", "- ", "hello world")</code> return? (A) hello-world (B) hello---world (C) hello world (D) Error Circle your answer: (A) (B) (C) (D)	Q8. What is the output of <code>print("abc" * 2 == "ab" * 3)</code>? (A) True (B) False (C) Error (D) None Circle your answer: (A) (B) (C) (D)

Part B — Predict the Output [6 × 2 = 12 marks]

Write exactly what each snippet prints. Write ERROR if it raises an exception.

Code Snippet	Your Output	Expected
<pre>def f(x, y=2, z=3): return x + y + z print(f(1, z=10))</pre>		
<pre>res = list(filter(lambda x: x%2==0, range(1,10))) print(res)</pre>		
<pre>def outer(x): def inner(y): return x + y return inner add5 = outer(5) print(add5(3))</pre>		
<pre>import re print(len(re.findall(r'[aeiou]', 'programming')))</pre>		
<pre>s = 'Hello World' print(s[::-1])</pre>		
<pre>def fact(n): return 1 if n<=1 else n*fact(n-1) print(fact(0), fact(1), fact(5))</pre>		

Part C — Code Execution Trace [2 × 5 = 10 marks]

Fill in the variable table step by step and write the final output.

Q1. Trace the execution — fill in the variable table and final output.

```
def mystery(s):
    if len(s) <= 1:
        return s
    return mystery(s[1:]) + s[0]

print(mystery("abc"))
```

Step	s	return value	Output

Part D — Fill in the Blanks [6 × 1 = 6 marks]

Complete each statement with the correct term, symbol, or value.

1. A lambda function is also called an _____ function.
2. `reduce(lambda a, b: a*b, [1, 2, 3, 4])` returns _____.
3. The `re._____` function checks for a match only at the BEGINNING of the string.
4. The string method _____ returns True if all characters are digits.
5. In Python, a function that calls itself is called a _____ function.
6. `ord("a") - ord("A")` equals _____.

Part E — Find & Fix the Error [3 × 4 = 12 marks]

Identify the bug(s) and write the corrected version of the code.

1. This memoized Fibonacci is supposed to be fast but crashes. Fix it.

```
cache = {}  
  
def fib(n):  
    if n in cache:  
        return cache[n]  
  
    if n <= 1:  
        return n  
  
    result = fib(n-1) + fib(n-2)  
  
    cache[n] = result  
  
    return result
```

Bug identified:

Corrected code:

2. The regex should extract all emails from text but misses some. Fix the pattern.

```
import re

text = "Send to admin@site.com or user.name@sub.org"

matches = re.findall(r"\w+@\w+\.\w+", text)

print(matches)
```

Bug identified:

Corrected code:

3. Fix: the function should modify the global counter.

```
counter = 0

def increment():

    counter += 1

    increment()

print(counter)
```

Bug identified:

Corrected code:

Part F — Write Programs [3 × 5 = 15 marks]

Attempt ALL three questions. Write clean, commented code.

Q1. Write a decorator function `@timer` that prints the time taken by any function to execute. Apply it to a function that computes the sum of 1 to n.

Hint: Use the time module: `import time`. Decorators wrap a function: `def timer(func): def wrapper(*args): ...`

Q2. Write a function that takes a string and returns a dictionary with the count of each character type: letters, digits, spaces, and others.

Hint: Use `str.isalpha()`, `str.isdigit()`, `str.isspace()` methods.

Q3. Write a function `flatten(lst)` that takes a nested list of any depth and returns a flat list using recursion.

Hint: Check if each element is a list using `isinstance(item, list)`. If yes, recurse; otherwise, keep the element.

UNIT 4 — File Handling and OOP

Advanced difficulty · 8 MCQs · 6 Output Predictions · 2 Code Traces · 6 Fill-in-Blanks · 3 Error Fixes · 3 Write Programs

Section	Marks	Your Score	Remarks
A — MCQ	16		
B — Output Predict	12		
C — Code Trace	10		
D — Fill Blanks	6		
E — Error Fix	12		
F — Write Programs	15		
TOTAL	71		

Part A — Multiple Choice [8 × 2 = 16 marks]

Circle the correct answer. No partial credit.

Q1. What is the output of with
`open("f.txt", "w") as f: f.write("hi");`
`print(f.closed)?`
 (A) False (B) True (C) Error (D) None
 Circle your answer: (A) (B) (C) (D)

Q2. Which OOP concept does `def speak(self)` in both Animal and Dog classes demonstrate?
 (A) Encapsulation (B) Abstraction (C) Method overriding (Polymorphism) (D) Composition
 Circle your answer: (A) (B) (C) (D)

Q3. What is the MRO (Method Resolution Order) for:
`class C(A, B)?`
 (A) $C \rightarrow B \rightarrow A$ (B) $C \rightarrow A \rightarrow B$ (C) $A \rightarrow B \rightarrow C$ (D) $B \rightarrow A \rightarrow C$
 Circle your answer: (A) (B) (C) (D)

Q4. What does `__slots__` do in a class?
 (A) Defines private methods (B) Restricts allowed instance attributes, saving memory (C) Creates read-only attributes (D) Overloads the `[]` operator
 Circle your answer: (A) (B) (C) (D)

Q5. What is the output of `f =`
`open("x.txt", "w"); f.write("abc");`
`f.seek(1); print(f.read())?`
 (A) bc (B) abc (C) Error (D) Empty string
 Circle your answer: (A) (B) (C) (D)

Q6. Which special method makes an object callable like a function?
 (A) `__run__` (B) `__call__` (C) `__exec__` (D) `__invoke__`
 Circle your answer: (A) (B) (C) (D)

Q7. What does `isinstance(True, int)` return?
 (A) False (B) True (C) Error (D) None
 Circle your answer: (A) (B) (C) (D)

Q8. Which method is called when you use `len(obj)`?
 (A) `__size__` (B) `__count__` (C) `__length__` (D) `__len__`
 Circle your answer: (A) (B) (C) (D)

Part B — Predict the Output [6 × 2 = 12 marks]

Write exactly what each snippet prints. Write ERROR if it raises an exception.

Code Snippet	Your Output	Expected
<pre>class A: x = 10 class B(A): pass B.x = 20 print(A.x, B.x)</pre>		
<pre>class C: def __init__(self): self.n = 0 def __add__(self, o): r = C() r.n = self.n + o.n return r a = C(); a.n = 3 b = C(); b.n = 4 print((a+b).n)</pre>		
<pre>class Animal: def sound(self): return "generic" class Cat(Animal): def sound(self): return "meow" animals = [Animal(), Cat()] for a in animals: print(a.sound())</pre>		
<pre>with open("tmp.txt","w") as f: f.write("Hello\nWorld") with open("tmp.txt","r") as f: lines = f.readlines() print(len(lines), lines[1].strip())</pre>		
<pre>class Counter: count = 0 def __init__(self): Counter.count += 1 Counter(); Counter(); Counter() print(Counter.count)</pre>		
<pre>class Node: def __init__(self,v): self.v=v self.next=None a=Node(1); b=Node(2) a.next=b print(a.next.v)</pre>		

Part C — Code Execution Trace [2 × 5 = 10 marks]

Fill in the variable table step by step and write the final output.

Q1. Trace the execution — fill in the variable table and final output.

```
class Stack:
    def __init__(self):
        self.data = []
    def push(self, v):
        self.data.append(v)
    def pop(self):
        return self.data.pop()
    def peek(self):
        return self.data[-1]

s = Stack()
s.push(10)
s.push(20)
s.push(30)
print(s.pop())
print(s.peek())
```

Step	s.data	output	Output

Part D — Fill in the Blanks [6 × 1 = 6 marks]

Complete each statement with the correct term, symbol, or value.

1. The _____ method is called automatically when an object is deleted or garbage collected.
2. To open a file for both reading and writing without truncating, use mode _____.
3. The _____ function returns True if an object is an instance of a given class or its subclasses.
4. Method _____ allows you to define what + does between two custom objects.
5. In multiple inheritance `class C(A, B)`, the method lookup order is called _____.

6. To make an attribute read-only using a property, use the _____ decorator.

Part E — Find & Fix the Error [3 × 4 = 12 marks]

Identify the bug(s) and write the corrected version of the code.

1. Fix the class so that the class variable "count" correctly tracks the number of instances.

```
class Person:
    count = 0
    def __init__(self, name):
        self.name = name
        count += 1 # bug here

p1 = Person("Raj")
p2 = Person("Priya")
print(Person.count)
```

Bug identified:

Corrected code:

2. The child class should override `__str__` but print still shows the parent version. Fix it.

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand
    def __str__(self):
        return f"Vehicle: {self.brand}"

class Car(Vehicle):
    def __init__(self, brand, model):
        super().__init__(brand)
        self.model = model
    def display(self): # wrong method name
        return f"Car: {self.brand} {self.model}"

c = Car("Toyota", "Camry")
print(c)
```

Bug identified:

Corrected code:

3. Fix the file-reading program that crashes when the file does not exist.

```
filename = input("Enter filename: ")  
f = open(filename, "r")  
content = f.read()  
f.close()  
print(content)
```

Bug identified:

Corrected code:

Part F — Write Programs [3 × 5 = 15 marks]

Attempt ALL three questions. Write clean, commented code.

Q1. Implement a `BankAccount` class with `deposit`, `withdraw`, and a transaction history list. Add a method `statement()` that prints all transactions.

Hint: Store transactions as tuples (type, amount). Use `@property` for balance.

Q2. Write a Python program that reads a text file and writes a new file with: (a) line numbers added, (b) all words capitalized, (c) a summary line at the end with word count and line count.

Hint: Use `enumerate()` for line numbers. `line.title()` capitalizes words. Track counts while iterating.

Q3. Implement a generic Stack class with `push`, `pop`, `peek`, `is_empty`, and `__len__`. Then use it to check if a string of brackets is balanced.

Hint: Balanced brackets: every opening bracket has a matching closing bracket in correct order. Use a dict for pairs.

— ANSWER KEY — Do not open until test is complete —

ANSWER KEY — Unit 1: Introduction to Python

Part A — MCQ Answers

Q1. What is the output of <code>bool("")</code> or <code>bool(0)</code> or <code>bool([])</code>? (A) True (B) False (C) None (D) Error Ans: (B) False All three are falsy values in Python. "or" on all False values returns the last one, which is <code>[]</code> — a falsy value evaluated as False.	Q2. Which expression evaluates to True? (A) not not 0 (B) not not "" (C) not not [] (D) not not 1 Ans: (D) not not 1 not not 1 → not False → True. All others are falsy so not not falsy → False.
Q3. What does <code>print(0.1 + 0.2 == 0.3)</code> print? (A) True (B) False (C) 0.3 (D) Error Ans: (B) False Floating point representation issues mean <code>0.1+0.2</code> is <code>0.30000000000000004</code>, not exactly 0.3.	Q4. What is the value of x after: <code>x = 5; x /= 2; x **= 2</code>? (A) 6 (B) 4 (C) 2 (D) 9 Ans: (D) 9 <code>x = 5 // 2 = 2</code>, then <code>x = 2**2 = 4</code>. Wait — <code>5//2=2</code>, <code>2**2=4</code>. Answer is 4.
Q5. Which is the correct way to swap two variables without a temp variable? (A) <code>a,b = b,a</code> (B) <code>a = b; b = a</code> (C) <code>swap(a,b)</code> (D) <code>a ^= b ^= a</code> Ans: (A) a,b = b,a Python supports tuple unpacking for swapping: <code>a, b = b, a</code> — both sides evaluated before assignment.	Q6. What is the output of <code>print(type(1/1))</code>? (A) <code><class 'int'></code> (B) <code><class 'float'></code> (C) <code><class 'double'></code> (D) Error Ans: (B) <class 'float'> In Python 3, <code>/</code> always returns float, even when dividing two integers. <code>1/1 = 1.0</code>.
Q7. What is the output of <code>x=10; print(x>5>3)</code>? (A) True (B) False (C) Error (D) TrueTrue Ans: (A) True Python supports chained comparisons. <code>x>5>3</code> is equivalent to <code>x>5</code> and <code>5>3</code> → True and True → True.	Q8. Which statement about <code>pass</code> is correct? (A) Exits the loop (B) Skips one iteration (C) Does nothing — placeholder (D) Returns None Ans: (C) Does nothing — placeholder <code>pass</code> is a no-op statement used as a placeholder where Python syntax requires a statement but no action is needed.

Part B — Output Prediction Answers

Code Snippet	Your Output	Correct Output
<pre>x = 15 print(x > 10 > 5)</pre>		True
<pre>a = b = c = 5 print(a, b, c)</pre>		5 5 5
<pre>x = 10 x, y = x + 1, x - 1 print(x, y)</pre>		11 9
<pre>print(int(True) + int(False) + 1)</pre>		2

Code Snippet	Your Output	Correct Output
<pre>x = '5' y = int(x) ** 2 print(type(y), y)</pre>		25
<pre>x = 5 if x % 2: print("odd") else: print("even")</pre>		odd

Part C — Code Trace Answers

Q1. Trace the execution — fill in the variable table and final output.

<pre>x = 1 for i in range(4): x = x * (i + 1) print(x)</pre>			
Step	i	x	Output
start	-	1	
i=0	0	1*1=1	
i=1	1	1*2=2	
i=2	2	2*3=6	
i=3	3	6*4=24	print: 24

Q2. Trace the execution — fill in the variable table and final output.

<pre>a, b = 6, 4 while b != 0: a, b = b, a % b print(a)</pre>			
Step	a	b	Output
start	6	4	
iter 1	4	6%4=2	
iter 2	2	4%2=0	
end	2	0	print: 2

Part D — Fill in the Blanks Answers

1. The operator **%** gives the remainder of division.

2. In Python, **2.5** is the result of $10 / 4$ (not $10 // 4$).
3. Python uses **indentation** instead of curly braces to define code blocks.
4. The **int()** function converts a string to an integer.
5. A chained comparison like $1 < x < 10$ is equivalent to **$1 < x$ and $x < 10$** .
6. The built-in **type()** function returns the type of a variable.

Part E — Error Fix Answers

1. This program should compute x squared but crashes. Find and fix all bugs.

```
x = input("Enter number: ")
result = x ^ 2
print("Square is: " result)
```

Bug identified:

Two bugs: (1) input() returns str — must cast to int(). (2) ^ is XOR not power — use **. (3) Missing comma in print.

Corrected code:

```
x = int(input("Enter number: ")) result = x ** 2 print("Square is:", result)
```

2. The if-elif chain should print grade but always prints "Fail". Fix it.

```
marks = int(input("Marks: "))
if marks > 90:
    print("A")
if marks > 75:
    print("B")
if marks > 60:
    print("C")
else:
    print("Fail")
```

Bug identified:

Using separate if statements instead of if-elif-else. Each condition is checked independently so marks=95 triggers all.

Corrected code:

```
marks = int(input("Marks: ")) if marks >= 90: print("A") elif marks >= 75: print("B")
elif marks >= 60: print("C") else: print("Fail")
```

3. Fix the infinite loop.

```
n = 10

while n > 0:

    print(n)
```

Bug identified:

Missing decrement — n never changes so condition $n > 0$ is always True.

Corrected code:

```
n = 10 while n > 0: print(n) n -= 1 # decrement n
```

Part F — Model Answers

Q1. Write a program that takes a positive integer n and prints the multiplication table of n from 1 to 10, formatted as " $n \times i = \text{result}$ ".

Hint: Use a for loop with `range(1, 11)`. f-strings or `format()` make the output clean.

```
n = int(input("Enter number: "))

for i in range(1, 11):

    print(f"{n} x {i} = {n * i}")
```

Q2. Write a program to check if a number is a **perfect number**. A perfect number equals the sum of its proper divisors (e.g. $6 = 1+2+3$).

Hint: Proper divisors are all divisors excluding the number itself. Use `%` to find divisors.

```
n = int(input("Enter number: "))

total = sum(i for i in range(1, n) if n % i == 0)

if total == n:

    print(f"{n} is a perfect number")

else:

    print(f"{n} is not a perfect number")
```

Q3. Write a program that prints all prime numbers between 1 and 100 using nested loops. Count them too.

Hint: A prime has no divisors other than 1 and itself. Check divisibility from 2 to $\sqrt{n}$.

```
count = 0
for n in range(2, 101):
    is_prime = True
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            is_prime = False
            break
    if is_prime:
        print(n, end=" ")
    count += 1
print(f"\nTotal primes: {count}")
```


ANSWER KEY — Unit 2: Loops and Data Structures

Part A — MCQ Answers

Q1. What is the output of <code>L = [1, 2, 3]; L[1:2] = [10, 20]; print(L)</code>? (A) [1,10,20,3] (B) [1,[10,20],3] (C) [10,20,3] (D) Error Ans: (A) [1,10,20,3] Slice assignment replaces the slice [2] (just element at index 1) with [10,20]. Result: [1,10,20,3].	Q2. What does <code>list(zip([1, 2], [3, 4], [5, 6]))</code> produce? (A) [[1,3,5],[2,4,6]] (B) [(1,3,5),(2,4,6)] (C) [(1,2),(3,4),(5,6)] (D) Error Ans: (B) [(1,3,5),(2,4,6)] <code>zip()</code> pairs elements by position across iterables, returning tuples: [(1,3,5),(2,4,6)].
Q3. Which operation on a frozenset raises an <code>AttributeError</code>? (A) <code>len(fs)</code> (B) <code>fs {1}</code> (C) <code>fs.add(4)</code> (D) <code>4 in fs</code> Ans: (C) <code>fs.add(4)</code> <code>frozenset</code> is immutable — <code>.add()</code> and <code>.remove()</code> do not exist. Operators like <code> </code> that return NEW <code>frozensets</code> are fine.	Q4. What is the time complexity of checking membership in a set (<code>x in s</code>)? (A) $O(n)$ (B) $O(\log n)$ (C) $O(1)$ average (D) $O(n^2)$ Ans: (C) $O(1)$ average Sets use hash tables. Average membership check is $O(1)$, unlike lists which are $O(n)$.
Q5. What does <code>{k:v for k,v in d.items() if v > 5}</code> do? (A) Filters keys (B) Filters by value > 5 (C) Inverts the dict (D) Sorts the dict Ans: (B) Filters by value > 5 Dict comprehension with a condition — keeps only key-value pairs where the value is greater than 5.	Q6. What is the output of <code>d = {}; d[1] = d[1] + 1 if 1 in d else 1; print(d)</code>? (A) Error (B) {1: 0} (C) {1: 1} (D) {1: 2} Ans: (C) {1: 1} Key 1 is not in d, so the else branch executes: <code>d[1] = 1</code>. Result: {1: 1}.
Q7. Which list method has $O(1)$ complexity? (A) <code>list.insert(0, x)</code> (B) <code>list.pop()</code> (C) <code>list.remove(x)</code> (D) <code>list.index(x)</code> Ans: (B) <code>list.pop()</code> <code>list.pop()</code> removes the last element in $O(1)$. Inserting/removing from the front or searching are $O(n)$.	Q8. What does <code>sorted({3, 1, 4, 1, 5, 9})</code> return? (A) Error — sets are unordered (B) {1,3,4,5,9} (C) [1,3,4,5,9] (D) (1,3,4,5,9) Ans: (C) [1,3,4,5,9] <code>sorted()</code> works on any iterable and returns a sorted LIST. Duplicates are removed because input is a set.

Part B — Output Prediction Answers

Code Snippet	Your Output	Correct Output
<pre>L = [1, 2, 3, 4, 5] print(L[::2])</pre>		[1, 3, 5]
<pre>a = [1, 2] b = a b.append(3) print(a)</pre>		[1, 2, 3]
<pre>d = {"a":1, "b":2} d.update({"b":10, "c":3}) print(sum(d.values()))</pre>		14

Code Snippet	Your Output	Correct Output
<pre>L = [3,1,2] print(sorted(L), L)</pre>		[1, 2, 3] [3, 1, 2]
<pre>A={1,2,3};B={2,3,4} print(len(A^B))</pre>		2
<pre>L = [[1,2],[3,4]] print(L[1][0])</pre>		3

Part C — Code Trace Answers

Q1. Trace the execution — fill in the variable table and final output.

```
nums = [5, 3, 8, 1, 9, 2]
maxi = nums[0]
for n in nums:
    if n > maxi:
        maxi = n
print(maxi)
```

Step	n	maxi	Output
start	-	5	
n=5	5	5	
n=3	3	5	
n=8	8	8	
n=1	1	8	
n=9	9	9	
n=2	2	9	print: 9

Part D — Fill in the Blanks Answers

- To copy a list without aliasing, use `b = a.copy()` or `b = a[:]`.
- `dict.items()` returns a view of **key-value** pairs.
- The **pop()** method removes and returns the last element of a list.
- A set is **unordered**, meaning you cannot index or slice it.
- `[x for x in range(10) if x%3==0]` produces **[0, 3, 6, 9]**.
- To merge two dicts in Python 3.9+, use the **| (pipe)** operator.

Part E — Error Fix Answers

1. The code should print a copy of the list doubled but has an aliasing bug.

```
original = [1, 2, 3]
copy = original
copy.append(4)
print("original:", original)
print("copy:", copy)
```

Bug identified:**copy = original creates an alias (both point to same list). Modifying copy also modifies original.****Corrected code:**

```
original = [1, 2, 3] copy = original.copy() # or original[:]
copy.append(4)
print("original:", original) # [1,2,3]
print("copy:", copy) # [1,2,3,4]
```

2. This dictionary word-frequency counter crashes. Fix it.

```
words = ["cat", "dog", "cat", "bird", "dog", "cat"]
freq = {}
for w in words:
    freq[w] += 1
print(freq)
```

Bug identified:**freq[w] += 1 fails with KeyError when the key does not exist yet.****Corrected code:**

```
words = ["cat", "dog", "cat", "bird", "dog", "cat"]
freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1
print(freq) # {"cat":3, "dog":2, "bird":1}
```

3. Fix: the program should find common elements between two lists efficiently.

```
A = [1,2,3,4,5]
B = [3,4,5,6,7]
common = []
for x in A:
    for y in B:
        if x == y:
            common.append(x)
print(common)
```

Bug identified: **$O(n^2)$ nested loop — inefficient. Better: use set intersection.****Corrected code:**

```
A = [1,2,3,4,5] B = [3,4,5,6,7] common = list(set(A) & set(B)) print(sorted(common)) #
[3, 4, 5]
```

Part F — Model Answers

Q1. Write a program that reads a sentence from the user and prints a dictionary showing the frequency of each word (case-insensitive, ignore punctuation).

Hint: Use `.lower()` and `.replace()` to normalize. Use `dict.get(word, 0) + 1` pattern.

```
sentence = input("Enter sentence: ").lower()
for ch in ".!?,;":
    sentence = sentence.replace(ch, "")
words = sentence.split()
freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1
for word, count in sorted(freq.items()):
    print(f"{word}: {count}")
```

Q2. Write a program that uses list comprehension to generate a 5x5 multiplication table stored as a 2D list, then prints it formatted.

Hint: Nested list comprehension: `[[i*j for j in range(1,6)] for i in range(1,6)]`

```
table = [[i*j for j in range(1,6)] for i in range(1,6)]
for row in table:
    print(" ".join(f"{v:3}" for v in row))
```

Q3. Write a program that takes a list of integers and partitions them into three lists: negatives, zeros, and positives — using a single pass through the list.

Hint: Use three separate lists and a single for loop. Do NOT use filter() for this question.

```
nums = list(map(int, input("Enter space-separated numbers: ").split()))
neg, zer, pos = [], [], []
for n in nums:
    if n < 0: neg.append(n)
    elif n == 0: zer.append(n)
    else: pos.append(n)
print("Negatives:", neg)
print("Zeros:", zer)
print("Positives:", pos)
```

ANSWER KEY — Unit 3: Functions and Strings

Part A — MCQ Answers

Q1. What is the output of <code>f = lambda *a: sum(a); print(f(1,2,3,4))</code>? (A) Error (B) 10 (C) [1,2,3,4] (D) None Ans: (B) 10 <i>*a collects all positional args into a tuple (1,2,3,4). sum((1,2,3,4)) = 10.</i>	Q2. What does <code>list(map(str, [1,2,3]))</code> return? (A) ['1','2','3'] (B) [1,2,3] (C) ('1','2','3') (D) Error Ans: (A) ['1','2','3'] <i>map(str, [1,2,3]) applies str() to each element. list() collects them: ['1','2','3'].</i>
Q3. How many times does <code>fib(5)</code> call itself (including the initial call) using naive recursion? (A) 5 (B) 9 (C) 15 (D) 25 Ans: (C) 15 <i>fib(5) → fib(4)+fib(3) → ... → 15 total calls. Naive recursion is exponential $O(2^n)$.</i>	Q4. What is the output of <code>"python"[1::2]</code>? (A) yh (B) pto (C) yto (D) phn Ans: (C) yto <i>Slice [1::2] starts at index 1 and takes every 2nd char: index 1=y, 3=h, 5=n → "yhn". Wait: p=0,y=1,t=2,h=3,o=4,n=5. [1::2] → y,h,n = "yhn".</i>
Q5. Which regex pattern matches a valid Indian mobile number (starts with 6-9, 10 digits)? (A) <code>r"\d{10}"</code> (B) <code>r"[6-9]\d{9}"</code> (C) <code>r"[0-9]{10}"</code> (D) <code>r"[6789]\d{10}"</code> Ans: (B) <code>r"[6-9]\d{9}"</code> <i>[6-9] matches the first digit (6,7,8 or 9). \d{9} matches exactly 9 more digits = 10 total.</i>	Q6. What is a closure in Python? (A) A function inside a class (B) An inner function that captures variables from its enclosing scope (C) A lambda with no arguments (D) A function that returns None Ans: (B) An inner function that captures variables from its enclosing scope <i>A closure is an inner function that remembers the variables from the enclosing scope even after that scope has finished executing.</i>
Q7. What does <code>re.sub(r"\s+", "-", "hello world")</code> return? (A) hello-world (B) hello---world (C) hello world (D) Error Ans: (A) hello-world <i>re.sub replaces one or more whitespace characters (\s+) with a single "-". "hello world" → "hello-world".</i>	Q8. What is the output of <code>print("abc" * 2 == "ab" * 3)</code>? (A) True (B) False (C) Error (D) None Ans: (B) False <i>"abc" * 2 = "abcabc" (6 chars), "ab" * 3 = "ababab" (6 chars). They are different strings → False.</i>

Part B — Output Prediction Answers

Code Snippet	Your Output	Correct Output
<pre>def f(x, y=2, z=3): return x + y + z print(f(1, z=10))</pre>		13
<pre>res = list(filter(lambda x: x%2==0, range(1,10))) print(res)</pre>		[2, 4, 6, 8]

Code Snippet	Your Output	Correct Output
<pre>def outer(x): def inner(y): return x + y return inner add5 = outer(5) print(add5(3))</pre>		8
<pre>import re print(len(re.findall(r'[aeiou]', 'programming')))</pre>		3
<pre>s = 'Hello World' print(s[::-1])</pre>		dlroW olleH
<pre>def fact(n): return 1 if n<=1 else n*fact(n-1) print(fact(0), fact(1), fact(5))</pre>		1 1 120

Part C — Code Trace Answers

Q1. Trace the execution — fill in the variable table and final output.

<pre>def mystery(s): if len(s) <= 1: return s return mystery(s[1:]) + s[0] print(mystery("abc"))</pre>			
Step	s	return value	Output
call 1	"abc"	mystery("bc") + "a"	
call 2	"bc"	mystery("c") + "b"	
call 3	"c"	"c" (base case)	
unwind 2	"bc"	"c"+"b" = "cb"	
unwind 1	"abc"	"cb"+"a" = "cba" → print: cba	

Part D — Fill in the Blanks Answers

1. A lambda function is also called an **anonymous** function.
2. `reduce(lambda a,b: a*b, [1, 2, 3, 4])` returns **24**.
3. The `re.match()` function checks for a match only at the BEGINNING of the string.
4. The string method `isdigit()` returns True if all characters are digits.
5. In Python, a function that calls itself is called a **recursive** function.
6. `ord("a") - ord("A")` equals **32**.

Part E — Error Fix Answers

1. This memoized Fibonacci is supposed to be fast but crashes. Fix it.

```
cache = {}

def fib(n):
    if n in cache:
        return cache[n]
    if n <= 1:
        return n
    result = fib(n-1) + fib(n-2)
    cache[n] = result
    return result
```

Bug identified:

return result is at wrong indentation — it is outside the function. Should be indented inside def fib().

Corrected code:

```
cache = {}
def fib(n):
    if n in cache:
        return cache[n]
    if n <= 1:
        return n
    result = fib(n-1) + fib(n-2)
    cache[n] = result
    return result # must be inside function
```

2. The regex should extract all emails from text but misses some. Fix the pattern.

```
import re

text = "Send to admin@site.com or user.name@sub.org"

matches = re.findall(r"\w+@\w+\.\w+", text)

print(matches)
```

Bug identified:

\w+ does not match dots in local part (user.name) or subdomains. Use [lw.]+ for both parts.

Corrected code:

```
import re
text = "Send to admin@site.com or user.name@sub.org"
matches = re.findall(r"[lw.]+\@[lw.]+\.\w+", text)
print(matches)
```


3. Fix: the function should modify the global counter.

```
counter = 0

def increment():

    counter += 1

increment()

print(counter)
```

Bug identified:

counter += 1 creates a local variable. Python raises UnboundLocalError. Must declare global counter.

Corrected code:

```
counter = 0
def increment():
    global counter
    counter += 1
increment()
print(counter) # 1
```

Part F — Model Answers

Q1. Write a decorator function @timer that prints the time taken by any function to execute. Apply it to a function that computes the sum of 1 to n.

Hint: Use the time module: import time. Decorators wrap a function: def timer(func): def wrapper(*args): ...

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end-start:.4f}s")
        return result
    return wrapper

@timer
def sum_to_n(n):
    return sum(range(1, n+1))

print(sum_to_n(1000000))
```

Q2. Write a function that takes a string and returns a dictionary with the count of each character type: letters, digits, spaces, and others.

Hint: Use `str.isalpha()`, `str.isdigit()`, `str.isspace()` methods.

```
def char_count(s):
    result = {"letters":0, "digits":0, "spaces":0, "others":0}
    for ch in s:
        if ch.isalpha():
            result["letters"] += 1
        elif ch.isdigit():
            result["digits"] += 1
        elif ch.isspace():
            result["spaces"] += 1
        else:
            result["others"] += 1
    return result

print(char_count("Hello, World! 123"))
```

Q3. Write a function `flatten(lst)` that takes a nested list of any depth and returns a flat list using recursion.

Hint: Check if each element is a list using `isinstance(item, list)`. If yes, recurse; otherwise, keep the element.

```
def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result

print(flatten([1, [2, [3, 4]], [5, 6]])) # [1, 2, 3, 4, 5, 6]
```

ANSWER KEY — Unit 4: File Handling and OOP

Part A — MCQ Answers

Q1. What is the output of with
open("f.txt","w") as f: f.write("hi");
print(f.closed)?

(A) False (B) True (C) Error (D) None

Ans: (B) True

After the with block exits, the file is automatically closed.
f.closed returns True.

Q2. Which OOP concept does def speak(self) in both Animal and Dog classes demonstrate?

(A) Encapsulation (B) Abstraction (C) Method overriding (Polymorphism) (D) Composition

Ans: (C) Method overriding (Polymorphism)

Defining the same method in a child class replaces the parent version — this is method overriding, a form of polymorphism.

Q3. What is the MRO (Method Resolution Order) for:
class C(A, B)?

(A) C → B → A (B) C → A → B (C) A → B → C (D) B → A → C

Ans: (B) C → A → B

Python MRO follows C3 linearization. For C(A,B) it is C → A → B → object (left-to-right depth-first).

Q4. What does __slots__ do in a class?

(A) Defines private methods (B) Restricts allowed instance attributes, saving memory (C) Creates read-only attributes (D) Overloads the [] operator

Ans: (B) Restricts allowed instance attributes, saving memory

__slots__ = ["x","y"] prevents creating arbitrary instance attributes and reduces memory by avoiding __dict__.

Q5. What is the output of f =
open("x.txt", "w"); f.write("abc");
f.seek(1); print(f.read())?

(A) bc (B) abc (C) Error (D) Empty string

Ans: (C) Error

File opened in "w" mode — read() raises
UnsupportedOperation. Use "r+" to read after writing.

Q6. Which special method makes an object callable like a function?

(A) __run__ (B) __call__ (C) __exec__ (D) __invoke__

Ans: (B) __call__

Defining __call__(self, ...) allows an object to be called using
obj() syntax.

Q7. What does isinstance(True, int) return?

(A) False (B) True (C) Error (D) None

Ans: (B) True

In Python, bool is a subclass of int. True and False are
instances of both bool and int.

Q8. Which method is called when you use
len(obj)?

(A) __size__ (B) __count__ (C) __length__ (D) __len__

Ans: (D) __len__

Python calls __len__(self) when len() is used on an object. It
must return a non-negative integer.

Part B — Output Prediction Answers

Code Snippet	Your Output	Correct Output
<pre>class A: x = 10 class B(A): pass B.x = 20 print(A.x, B.x)</pre>		10 20

Code Snippet	Your Output	Correct Output
<pre>class C: def __init__(self): self.n = 0 def __add__(self, o): r = C() r.n = self.n + o.n return r a = C(); a.n = 3 b = C(); b.n = 4 print((a+b).n)</pre>		7
<pre>class Animal: def sound(self): return "generic" class Cat(Animal): def sound(self): return "meow" animals = [Animal(), Cat()] for a in animals: print(a.sound())</pre>		generic meow
<pre>with open("tmp.txt","w") as f: f.write("Hello\nWorld") with open("tmp.txt","r") as f: lines = f.readlines() print(len(lines), lines[1].strip())</pre>		2 World
<pre>class Counter: count = 0 def __init__(self): Counter.count += 1 Counter(); Counter(); Counter() print(Counter.count)</pre>		3
<pre>class Node: def __init__(self,v): self.v=v self.next=None a=Node(1); b=Node(2) a.next=b print(a.next.v)</pre>		2

Part C — Code Trace Answers

Q1. Trace the execution — fill in the variable table and final output.

```

class Stack:
    def __init__(self):
        self.data = []
    def push(self, v):
        self.data.append(v)
    def pop(self):
        return self.data.pop()
    def peek(self):
        return self.data[-1]

s = Stack()
s.push(10)
s.push(20)
s.push(30)
print(s.pop())
print(s.peek())

```

Step	s.data	output	Output
__init__	[]		
push(10)	[10]		
push(20)	[10,20]		
push(30)	[10,20,30]		
pop()	[10,20]	print: 30	
peek()	[10,20]	print: 20	

Part D — Fill in the Blanks Answers

1. The `__del__()` method is called automatically when an object is deleted or garbage collected.
2. To open a file for both reading and writing without truncating, use mode `"r+"`.
3. The `isinstance()` function returns True if an object is an instance of a given class or its subclasses.
4. Method `__add__()` allows you to define what + does between two custom objects.
5. In multiple inheritance `class C(A, B)`, the method lookup order is called **MRO (Method Resolution Order)**.
6. To make an attribute read-only using a property, use the `@property` decorator.

Part E — Error Fix Answers

1. Fix the class so that the class variable "count" correctly tracks the number of instances.

```
class Person:
    count = 0
    def __init__(self, name):
        self.name = name
        count += 1 # bug here

p1 = Person("Raj")
p2 = Person("Priya")
print(Person.count)
```

Bug identified:

count += 1 refers to a local variable. Should use Person.count += 1 or self.__class__.count += 1.

Corrected code:

```
class Person: count = 0
    def __init__(self, name): self.name = name
    Person.count += 1 # reference class variable
p1 = Person("Raj")
p2 = Person("Priya")
print(Person.count) # 2
```

2. The child class should override `__str__` but print still shows the parent version. Fix it.

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand
    def __str__(self):
        return f"Vehicle: {self.brand}"

class Car(Vehicle):
    def __init__(self, brand, model):
        super().__init__(brand)
        self.model = model
    def display(self): # wrong method name
        return f"Car: {self.brand} {self.model}"

c = Car("Toyota", "Camry")
print(c)
```

Bug identified:**Child defines `display()` but not `__str__()`. `print(c)` calls `__str__` which uses parent version.****Corrected code:**

```
class Car(Vehicle):
    def __init__(self, brand, model):
        super().__init__(brand)
        self.model = model
    def __str__(self):
        # must be __str__ not display
        return f"Car: {self.brand} {self.model}"
c = Car("Toyota", "Camry")
print(c) # Car: Toyota Camry
```

3. Fix the file-reading program that crashes when the file does not exist.

```
filename = input("Enter filename: ")
f = open(filename, "r")
content = f.read()
f.close()
print(content)
```

Bug identified:**No error handling — crashes with `FileNotFoundError` if file missing. Also not using `with` statement.****Corrected code:**

```
filename = input("Enter filename: ")
try:
    with open(filename, "r") as f:
        content = f.read()
    print(content)
except FileNotFoundError:
    print(f"Error: {filename} not found")
```

Part F — Model Answers

Q1. Implement a `BankAccount` class with `deposit`, `withdraw`, and a transaction history list. Add a method `statement()` that prints all transactions.

Hint: Store transactions as tuples (type, amount). Use `@property` for balance.

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.__balance = balance
        self.__history = []

    @property
    def balance(self):
        return self.__balance

    def deposit(self, amt):
        self.__balance += amt
        self.__history.append(("DEPOSIT", amt))

    def withdraw(self, amt):
        if amt > self.__balance:
            print("Insufficient funds")
            return
        self.__balance -= amt
        self.__history.append(("WITHDRAW", amt))

    def statement(self):
        print(f"Account: {self.owner}")
        for typ, amt in self.__history:
            print(f" {typ}: Rs.{amt}")
        print(f" Balance: Rs.{self.__balance}")

acc = BankAccount("Raj", 1000)
acc.deposit(500)
acc.withdraw(200)
acc.statement()
```


Q2. Write a Python program that reads a text file and writes a new file with: (a) line numbers added, (b) all words capitalized, (c) a summary line at the end with word count and line count.

Hint: Use `enumerate()` for line numbers. `line.title()` capitalizes words. Track counts while iterating.

```
input_file = "input.txt"
output_file = "output.txt"

with open(input_file, "r") as f:
    lines = f.readlines()

total_words = 0
with open(output_file, "w") as f:
    for i, line in enumerate(lines, 1):
        processed = line.title()
        f.write(f"{i:3}. {processed}")
        total_words += len(line.split())
    f.write(f"\n--- Summary: {len(lines)} lines, {total_words} words ---")

print("Done. Output written to", output_file)
```

Q3. Implement a generic Stack class with push, pop, peek, is_empty, and __len__. Then use it to check if a string of brackets is balanced.

Hint: Balanced brackets: every opening bracket has a matching closing bracket in correct order. Use a dict for pairs.

```
class Stack:
    def __init__(self):
        self.data = []
    def push(self, v): self.data.append(v)
    def pop(self): return self.data.pop() if self.data else None
    def peek(self): return self.data[-1] if self.data else None
    def is_empty(self): return len(self.data) == 0
    def __len__(self): return len(self.data)

def is_balanced(s):
    pairs = {"(": ")", "[": "]", "{": "}"}
    stack = Stack()
    for ch in s:
        if ch in "([{":
            stack.push(ch)
        elif ch in ")]}":
            if stack.is_empty() or stack.pop() != pairs[ch]:
                return False
    return stack.is_empty()

print(is_balanced("({[]})")) # True
print(is_balanced("{[[]}") # False
```